

Engenharia de Software

uma visão geral

Prof. Dr. Luã Marcelo Muriana
Engenharia e Elicitação de Requisitos
Engenharia de Software – 2025/2



O que é

Software?

O que é

Engenharia de Software?

“A engenharia de software abrange um processo, um conjunto de métodos (práticas) e um leque de ferramentas que possibilitam aos profissionais desenvolverem software de altíssima qualidade.”

(Pressman & Maxin, 2016, p.14)

Importância da Engenharia de Software

“A engenharia de software é importante porque nos capacita para o desenvolvimento de sistemas complexos dentro do prazo e com alta qualidade. Ela impõe disciplina a um trabalho que pode se tornar caótico, mas também permite que as pessoas produzam software de computador adaptado à sua abordagem, da maneira mais conveniente às suas necessidades.”

(Pressman & Maxin, 2016, p.14)

Importância da Engenharia de Software

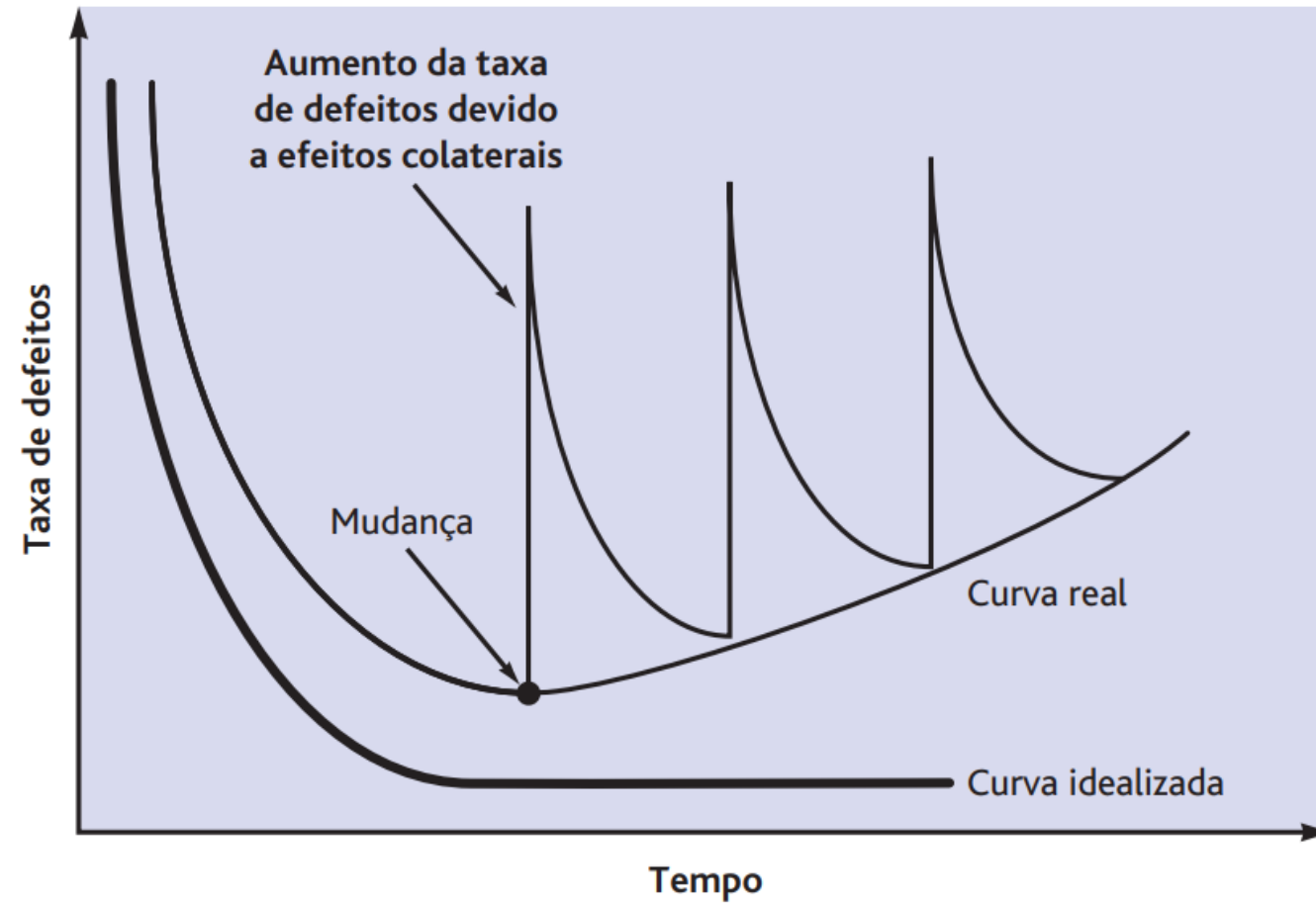


FIGURA 1.2 Curva de defeitos para software.

PRESSMAN, Roger S. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016. (p. 6)

Camadas da Engenharia de Software



FIGURA 2.1 Camadas da engenharia de software.

PRESSMAN, Roger S. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016. (p. 16)

Processos de Software

Metodologia do Processo

Estabelece a base para um processo de engenharia de software completo por meio de **atividades metodológicas** aplicáveis a todos os projetos de software.

Atividades genéricas de uma metodologia de processo

Comunicação

Planejamento

Modelagem

Construção

Entrega

Atividades de Apoio

- Controle e acompanhamento do projeto
- Administração de riscos
- Garantia da qualidade de software
- Revisões técnicas
- Medição
- Gerenciamento da configuração de software
- Gerenciamento da capacidade de reutilização
- Preparo e produção de artefatos de software

A prática da Engenharia de Software

- *Compreender o problema:* comunicação e análise
 - Quem tem interesse na solução do problema?
 - Quais são as incógnitas?
 - O problema pode ser compartimentalizado?
 - O problema pode ser representado graficamente?
- *Planejar uma solução:* modelagem e projeto de software
 - Você já viu problemas semelhantes anteriormente?
 - Algum problema semelhante já foi resolvido?
 - É possível definir subproblemas?
 - É possível representar uma solução de maneira que conduza a uma implementação efetiva?

A prática da Engenharia de Software

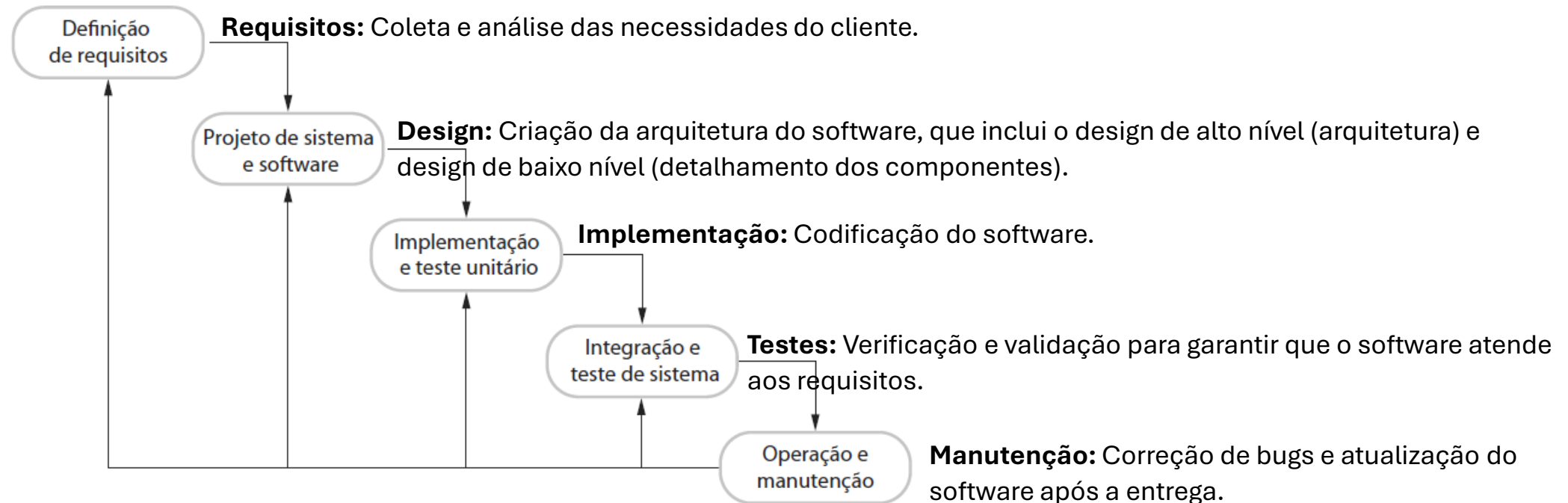
- *Executar o plano*: geração de código
 - A solução é adequada ao plano?
 - Todas as partes componentes da solução estão provavelmente corretas?
- *Examinar o resultado para ter precisão*: testes e garantia de qualidade
 - É possível testar cada parte componente da solução?
 - A solução produz resultados adequados aos dados, às funções e às características necessários?

Modelos de Processos de Software

Modelo Cascata

Abordagem tradicional de desenvolvimento de software que segue um processo linear e sequencial, onde cada fase deve ser concluída antes de iniciar a próxima

Figura 2.1 O modelo em cascata



Fonte: SOMMERVILLE, Ian. *Engenharia de Software*. 9. ed. São Paulo: Pearson Prentice Hall, 2011. (p. 20)

Modelo Incremental

Desenvolvimento em pequenos incrementos ou versões, onde cada incremento entrega parte das funcionalidades do sistema.

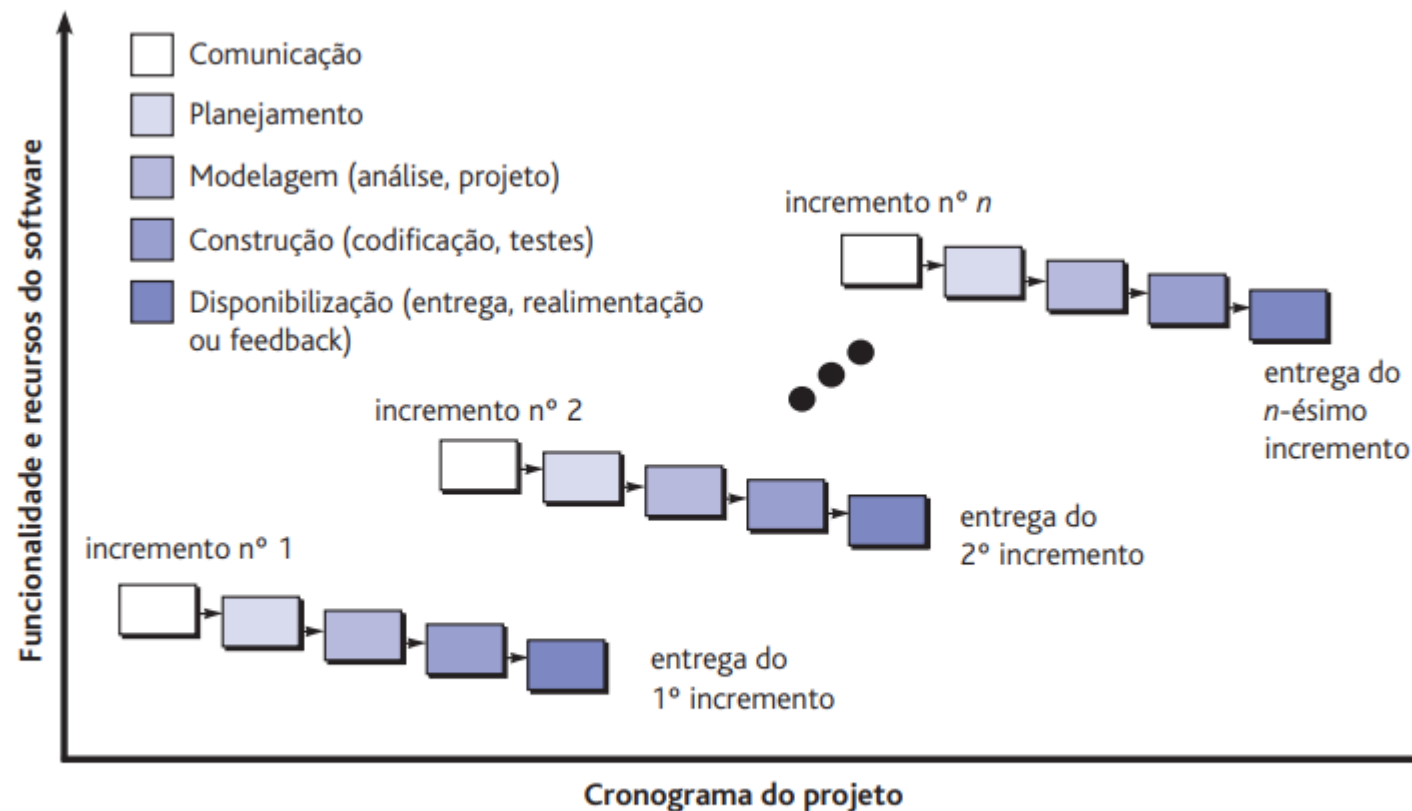


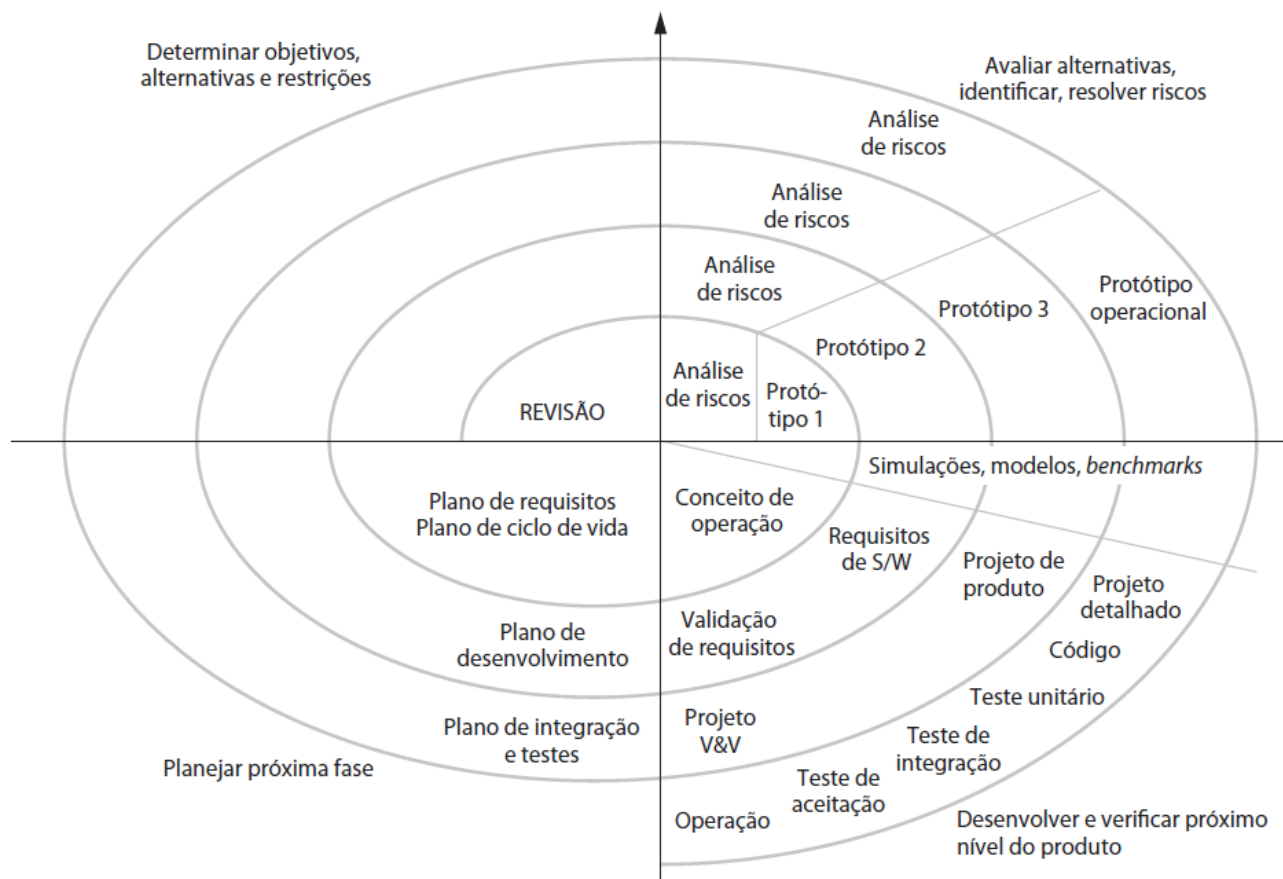
FIGURA 4.3 O modelo incremental.

PRESSMAN, Roger S. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016. (p. 44)

Modelo Espiral

Focado na avaliação de riscos e na repetição dos ciclos de desenvolvimento, combinando elementos do modelo cascata e prototipagem.

Figura 2.11 Modelo em espiral de processo de software de Boehm (©IEEE 1988)



Fonte: SOMMERVILLE, Ian. *Engenharia de Software*. 9. ed. São Paulo: Pearson Prentice Hall, 2011. (p. 33)

Metodologias Ágeis

Abordagem de desenvolvimento de software que enfatiza entregas incrementais, colaboração contínua e adaptação rápida a mudanças.

Scrum: Focado em entregas rápidas e iterativas, com revisões constantes e adaptações ao feedback do cliente.

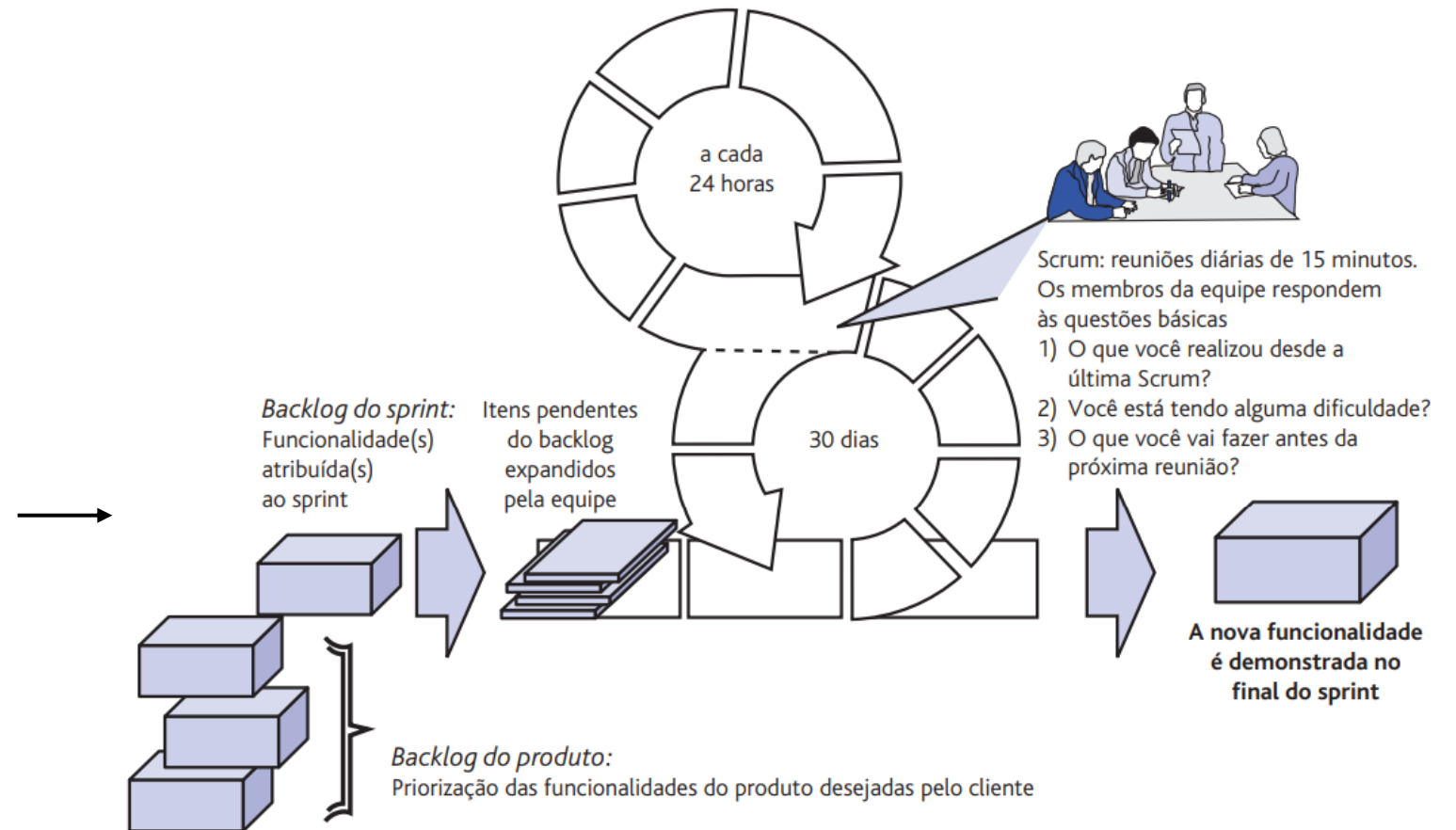


FIGURA 5.3 Fluxo do processo Scrum.

PRESSMAN, Roger S. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016. (p. 78)

Requisitos de Software

Requisitos de Software

Requisitos de software são descrições das funções, características, e restrições que um sistema de software deve possuir para atender às necessidades dos usuários

Tipos de Requisitos:

Funcionais: O que o software deve fazer (funcionalidades).

1. **Login de Usuário:** O sistema deve permitir que os usuários façam login usando um nome de usuário e senha.
2. **Geração de Relatórios:** O sistema deve gerar relatórios mensais de vendas em formato PDF.

Não Funcionais: Qualidades que o software deve ter (desempenho, segurança, usabilidade).

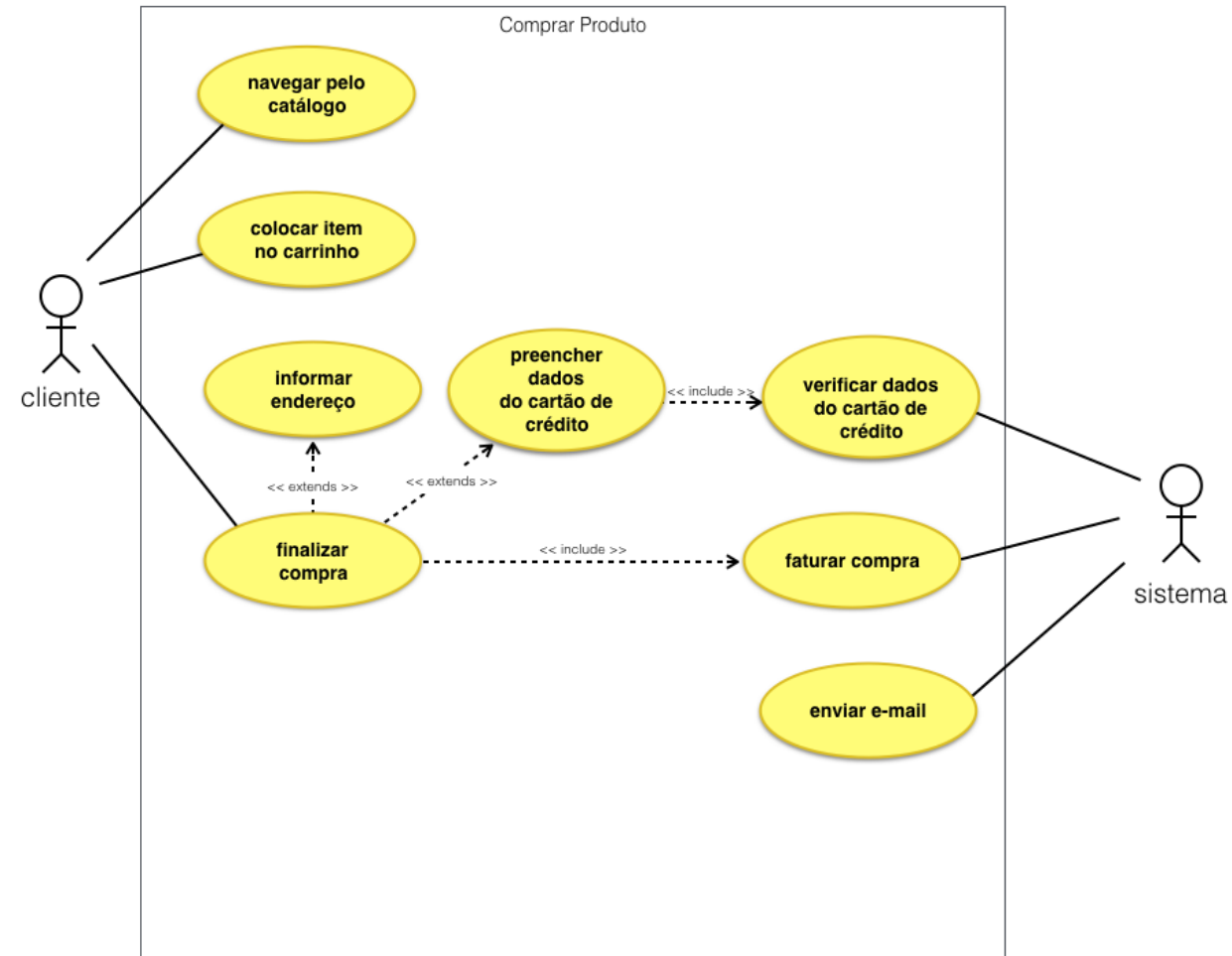
1. **Desempenho:** O sistema deve ser capaz de processar até 10.000 transações por minuto.
2. **Segurança:** Os dados dos usuários devem ser criptografados durante a transmissão e armazenamento.

Elicitação de Requisitos de Software

Levantamento de Requisitos:

Entrevistas, workshops, análise de documentos.

Análise e Modelagem: Criação de modelos (casos de uso, diagramas de sequência).



Fonte: <https://medium.com/operacionalti/uml-diagrama-de-casos-de-uso-29f4358ce4d5>

Validação de Requisitos

Validação de requisitos é o processo de garantir que os requisitos definidos para um sistema atendem às necessidades e expectativas dos stakeholders (clientes, usuários, etc.).

Importância

Requisitos bem validados reduzem o risco de retrabalho, custos adicionais e atrasos no projeto, além de garantir que o produto final será funcional e satisfatório.

Exemplos de Perguntas

- Todos os requisitos estão de acordo com os objetivos globais para o sistema ou produto?
Todos os requisitos foram especificados no nível de abstração apropriado?
- O requisito é realmente necessário ou representa um recurso adicional que talvez não seja essencial para o objetivo do sistema?
- Cada um dos requisitos é limitado e sem ambiguidade?
- Cada um dos requisitos possui atribuição?
- Algum dos requisitos conflita com outros requisitos?

PRESSMAN, Roger S. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016. (p. 161)

Design de Software

Design de Software (Projeto)

Design de Alto Nível (Arquitetura):

- Estrutura geral do sistema, definição de módulos e seus relacionamentos.
- **Padrões de Arquitetura:** MVC (Model-View-Controller), Microservices, Camadas.

Design de Baixo Nível:

- Detalhamento dos componentes e interfaces internas.
- **Padrões de Projeto (Design Patterns):** Factory, Singleton, Observer, Strategy.

Ferramentas de Design:

- Uso de diagramas UML (Unified Modeling Language), como diagramas de classes, sequência e atividades.



Implementação e Codificação

Implementação e Codificação

Princípios de Boa Codificação:

- **Código Limpo:** Facilidade de leitura e manutenção.
- **Reuso de Código:** Uso de bibliotecas e frameworks.
- **Controle de Versão:** Git e outras ferramentas para gerenciar mudanças no código.

Ferramentas de Desenvolvimento:

- IDEs, depuradores.

Testes de Software

Testes de Software

Teste de software é o processo de verificar e validar que um sistema de software funciona corretamente e atende aos requisitos especificados.

Importância

Garantir a qualidade e confiabilidade do software, identificando e corrigindo defeitos antes da implantação.

Tipos de Testes:

- **Unitário:** Testa partes individuais do código.
- **Integração:** Testa a interação entre diferentes partes do sistema.
- **Sistema:** Testa o sistema como um todo.
- **Aceitação:** Valida o sistema contra os requisitos do usuário final.

Automatização de Testes:

- Ferramentas e frameworks para automatizar a execução de testes.

Manutenção de Software

Manutenção de Software

Manutenção de software é o processo de modificar e atualizar um sistema de software após sua entrega inicial para corrigir falhas, melhorar desempenho ou adaptar o software a novos ambientes.

Importância

A manutenção pode consumir até 70% dos custos do ciclo de vida de um software, destacando a necessidade de abordá-la de forma eficiente.

Tipos de Manutenção de Software

Manutenção Corretiva: Corrigir erros e falhas identificados no software após sua entrega.

Exemplo: Correção de um bug que causa falha em determinada funcionalidade do sistema.

Manutenção Adaptativa: Modificar o software para que ele continue funcionando em um ambiente em mudança, como quando há atualizações no sistema operacional ou em outros softwares relacionados.

Exemplo: Ajuste do software para funcionar com uma nova versão do sistema operacional.

Manutenção Evolutiva: Adicionar novas funcionalidades ou melhorar as existentes para atender a novas necessidades do usuário.

Exemplo: Implementação de um novo módulo de relatórios em um sistema de gestão.

Manutenção Preventiva: Realizar modificações para melhorar a manutenibilidade do software e prevenir futuros problemas.

Exemplo: Refatoração de código para melhorar sua legibilidade e facilitar futuras manutenções.

Desafios da Eng. de Software

Complexidade Crescente dos Sistemas

Expansão e Integração de Sistemas:

- Os sistemas modernos estão se tornando cada vez mais complexos, envolvendo a integração de múltiplos serviços, plataformas e tecnologias.

Manutenção e Evolução:

- Sistemas antigos que precisam ser atualizados para integrar novas funcionalidades sem comprometer a estabilidade.

Exemplo Prático:

- Amazon ou Google gerenciam bilhões de transações diariamente, integrando sistemas de todo o mundo.

Necessidade de Segurança

Ameaças e Vulnerabilidades:

- Aumento das ameaças cibernéticas, destacando como a segurança deve ser uma prioridade desde a fase de design até a implementação.

Práticas de Engenharia Segura:

- Desenvolvimento seguro, a revisão contínua de código, e a necessidade de testes de penetração para identificar vulnerabilidades.

Exemplo Prático:

- Vazamentos de dados evidenciam a necessidade de engenharia de software focada na segurança.

Evolução Contínua da Tecnologia

Ritmo Rápido da Inovação:

- A rápida evolução tecnológica, com o surgimento de novas linguagens, frameworks e paradigmas, impõe a necessidade de aprendizado contínuo para os profissionais de software.

Desafios de Adoção:

- A adoção de novas tecnologias em projetos já em andamento, incluindo o risco de obsolescência tecnológica e a necessidade de requalificação da equipe.

Exemplo Prático:

- O crescimento de tecnologias como a computação em nuvem e o desenvolvimento de IA, que estão mudando significativamente as abordagens tradicionais de desenvolvimento de software.

Mas...

O que é Engenharia de Requisitos?

Engenharia de Requisitos

- A **Engenharia de Requisitos** é uma disciplina fundamental na área de desenvolvimento de software e sistemas. Ela se dedica a um conjunto de atividades que visam **identificar, analisar, documentar, validar e gerenciar** as necessidades e expectativas de todos os envolvidos (stakeholders) em um projeto.
- Em outras palavras, é a ponte entre o que os usuários e o negócio **precisam** e o que o sistema **deve fazer**. O principal objetivo é garantir que o produto final atenda de forma precisa e completa às expectativas e aos objetivos do projeto, minimizando retrabalho e problemas futuros.

Por que Engenharia de Requisitos?

- **Evita retrabalho e custos extras:** Definir claramente os requisitos no início do projeto evita que funcionalidades sejam construídas de forma errada ou incompletas, o que demandaria alterações caras e demoradas no futuro.
- **Melhora a comunicação:** Ela estabelece uma linguagem comum entre clientes, usuários, desenvolvedores e gerentes, garantindo que todos tenham o mesmo entendimento sobre o que será construído.
- **Aumenta a satisfação do cliente:** Ao compreender e atender as necessidades reais dos usuários, o produto final tende a ser mais útil e satisfatório.
- **Ajuda no planejamento:** Com requisitos bem definidos, é mais fácil estimar prazos, custos e recursos necessários para o projeto.
- **Reduz riscos:** Identificar e gerenciar riscos associados aos requisitos (como mudanças constantes ou ambiguidades) desde o início do projeto.
- **Garante a qualidade:** Requisitos claros e bem testados são a base para a construção de um software robusto e de alta qualidade.

Leituras Recomendadas

SOMMERVILLE, Ian. *Engenharia de Software*. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

- Cap. 2 – páginas 18 a 33;
- Cap. 3 – páginas 38 a 42;
- **Cap. 4 – todo (fundamental para a nossa disciplina);**
- Cap. 8 – todo (para quem quiser ter uma ideia de como são e quais são os testes de software);

PRESSMAN, Roger S. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016.

- Cap. 1 – páginas 1 a 6;
- Cap. 2 – páginas 14 a 21;
- Cap. 3 – páginas 30 a 34;
- Cap. 4 – páginas 40 a 49;
- Cap. 5 – páginas 66 a 72; e 78 a 79;
- **Cap. 8 – todo (fundamental para a nossa disciplina);**
- Cap. 22 – todo (para quem quiser ter uma ideia de como são e quais são os testes de software);

Referências

Fonte: SOMMERVILLE, Ian. *Engenharia de Software*. 9. ed. São Paulo: Pearson Prentice Hall, 2011. (p. 33)

PRESSMAN, Roger S. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016. (p. 161)